Reviewer Pack — Minimal Monomial Degree of Tropical Polynomial and SAT Claus…

Entry 15f91487a191 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 02:32:43 UTC

Minimal Monomial Degree of Tropical Polynomial and SAT Clause Entropy

Entry ID: `15f91487a191` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-05 02:32:43 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry **Field B** (complexity object): Boolean Satisfiability (SAT Clause Subsets)

Statement:

> For every Boolean satisfiability instance with n clauses, the minimal monomial degree d(n) of its associated tropical polynomial φ_n is linearly correlated with its clause entropy H(φ_n), such that d(n) = Θ(H(φ_n)).

Rationale (proposer's reasoning):

> Tropical geometry provides a framework to study algebraic properties of functions over the max-plus semiring, which can be used to analyze Boolean functions. The minimal monomial degree captures the complexity of these functions in terms of their tropical representation, while clause entropy quantifies the redundancy in the clause structure, suggesting that both measures could reveal underlying structures related to the satisfiability problem.

Taxonomy category: `TROPICAL_GEOOMETRY_SAT` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `eac515679b694d63`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For every Boolean satisfiability instance, if the Spearman's rank correlation coefficient between minimal monomial degree and clause entropy is ≥ 0.8 for at least 24 out of 30 seeds.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	SAFE
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "tropical geometry" AND "Boolean satisfiability" AND "minimal monomial degree" - "tropical polynomial" AND "SAT clause entropy" AND "correlation" - "degree of tropical polynomial" ~ "entropy of SAT clauses"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction
import sys

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_sat_instance(n):
        clauses = []
        for _ in range(n):
            literals = [random.choice([-1, 1]) * (i + 1) for i in range(random.randint(1, n))]
            clauses.append(literals)
        return clauses

    def tropical_polynomial(clauses):
        n = len(clauses[0])
        poly = [[-math.inf] * n for _ in range(n)]
        for clause in clauses:
            for literal in clause:
                i = abs(literal) - 1
                if literal > 0:
                    poly[i][i] = max(poly[i][i], 1)
                else:
                    for j in range(n):
                        poly[j][j] = max(poly[j][j], 1)
        return poly

    def minimal_monomial_degree(poly):
        n = len(poly)
        degree = 0
        for i in range(n):
            for j in range(n):
```

```
            if poly[i][j] != -math.inf:
                degree += 1
    return degree

def clause_entropy(clauses):
    n = len(clauses[0])
    entropy = 0
    for clause in clauses:
        p = Fraction(2 ** (-len(clause)), 2 ** n)
        entropy -= p * math.log(p, 2)
    return entropy

results = []
n_values = [5, 10, 15, 20, 30, 40]

for n in n_values:
    instances_tested = 0
    total_degree = 0
    total_entropy = 0

    while instances_tested < 30:
        clauses = generate_sat_instance(n)
        poly = tropical_polynomial(clauses)
        degree = minimal_monomial_degree(poly)
        entropy = clause_entropy(clauses)

        if degree is not None and entropy is not None:
            total_degree += degree
            total_entropy += entropy
            instances_tested += 1

    if instances_tested == 0:
        return {
            "metric_name": "Spearman's rank correlation coefficient",
            "metric_value": None,
            "instances_tested": 0,
            "n_max": n,
            "conjecture_holds": False,
            "counterexample": "not_enough_instances"
        }

    avg_degree = total_degree / instances_tested
    avg_entropy = total_entropy / instances_tested

    results.append((avg_degree, avg_entropy))

if len(results) == 0:
    return {
        "metric_name": "Spearman's rank correlation coefficient",
        "metric_value": None,
        "instances_tested": 0,
        "n_max": max(n_values),
        "conjecture_holds": False,
        "counterexample": "not_enough_instances"
    }
```

```
    n = len(results)
    degrees = [x[0] for x in results]
    entropies = [x[1] for x in results]

    rank_degrees = sorted(range(n), key=lambda i: degrees[i])
    rank_entropies = sorted(range(n), key=lambda i: entropies[i])

    spearman_coefficient = 0
    for i in range(n):
        spearman_coefficient += (rank_degrees[i] - rank_entropies[i]) ** 2

    n_max = max(n_values)
    conjecture_holds = spearman_coefficient <= 1.64 * math.sqrt(1 / n) if n >= 30 else False
    counterexample = "" if conjecture_holds else "not_enough_instances"

    return {
        "metric_name": "Spearman's rank correlation coefficient",
        "metric_value": spearman_coefficient,
        "instances_tested": sum(1 for _, _ in results),
        "n_max": n_max,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19,
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result["metric_value"])

    if len(results) == 0:
        print("RESULT: INCONCLUSIVE no_results")
    else:
        mean = sum(results) / len(results)
        std_dev = math.sqrt(sum((x - mean) ** 2 for x in results) / len(results))
        support_fraction = sum(1 for r in results if r is not None and r >= 0.8 * mean) / len(resu

        if support_fraction >= 0.8:
            print(f"RESULT: SUPPORTED mean={mean} std={std_dev} support_fraction={support_fractior
        else:
            first_failing_seed = next(seed for seed, result in zip(seeds, results) if result is no
            print(f"RESULT: FALSIFIED counterexample='not_enough_instances' first_failing_seed={fi
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_13c4bad2.py", line 131, in <module>
```

```
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_13c4bad2.py", line 68, in run_trial
    poly = tropical_polynomial(clauses)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_13c4bad2.py", line 35, in tropical_poly
    poly[i][i] = max(poly[i][i], 1)
                     ~~~~^^^
IndexError: list index out of range
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed before producing data, which means that the pre-registered support condition could not be unambiguously met. | next: Investigate and fix the crash in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11965
2	preregistration	ollama_remote	glm4:latest	0	0	9622
3	novelty	ollama_remote	glm4:latest	0	0	12169
4	novelty	ollama_remote	glm4:latest	0	0	9445
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14581
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11290
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11259
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14061
9	judge	ollama_remote	glm4:latest	0	0	12407

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 106799 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/15f91487a191.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/15f91487a191.jsonl` for verification of provenance

4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/15f91487a191.tar.gz` (if generated)